



FINAL YEAR ENGINEERING PROJECT

Smart Medicine Availability & Intelligent Janaushadhi Recommendation System

SOFTWARE ENGINEERING GUIDE

Project	Smart Medicine Availability & Intelligent Janaushadhi Recommendation System
Document	SOFTWARE ENGINEERING GUIDE
Version	v1.0.0
Date	02 July 2026

Professional Principles Applied in This Project

1. Clean Architecture

What is it?

Clean Architecture organises code into layers where each layer has a specific responsibility. Inner layers don't know about outer layers. Dependencies point inward.

How it is applied here

Presentation Layer → pages/, components/, layouts/
(React UI – renders what the user sees)

Application Layer → contexts/, hooks/
(Business logic – what happens when user acts)

Service Layer → services/
(API calls – how data is fetched/sent)

Infrastructure → config/, utils/, constants/
(Technical infrastructure – Axios, storage, routes)

A page component does not call Axios directly. It calls a service. The service calls Axios. When the backend URL changes, only `env.js` and `axiosClient.js` need updating.

2. Single Responsibility Principle (SRP)

What is it?

Every module, class, or function should have one, and only one, reason to change.

Examples in this project

File	Its Single Responsibility
<code>axiosClient.js</code>	Only HTTP configuration and interceptors
<code>authService.js</code>	Only auth API calls
<code>AuthContext.jsx</code>	Only authentication state
<code>LoginPage.jsx</code>	Only the login UI
<code>formatters.js</code>	Only data formatting functions
<code>validators.js</code>	Only validation functions
<code>storage.js</code>	Only localStorage operations

`LoginPage.jsx` does not call Axios. `authService.js` does not manage React state. Each file does one job.

Why it matters

If the API endpoint for login changes:

- With SRP: change one line in `authService.js`
- Without SRP: search through all components to find where the API was called

3. DRY Principle (Don't Repeat Yourself)

What is it?

Every piece of knowledge should have a single, unambiguous, authoritative representation.

How it is applied

Route paths — defined once:

```
// constants/routes.js
export const ROUTES = { USER: { DASHBOARD: '/dashboard' } }

// Used everywhere — never hardcoded
navigate(ROUTES.USER.DASHBOARD)
```

Badge component — written once, used everywhere:

- Search results: "In Stock"
- Inventory: "Low Stock"
- User table: "Admin"
- Medicine cards: "Generic"

All use the same `<Badge>` component.

Design tokens — defined once:

```
@theme { --color-primary-600: #2563eb; }
/* Used as Tailwind class: bg-primary-600, text-primary-600, border-primary-600 */
```

Change the brand colour in one place → updates everywhere.

4. Separation of Concerns

What is it?

Different aspects of the application should be handled in different places. UI logic, business logic, and data access should not be mixed.

In this project

UI Logic	→ components/ (How things look and how users interact)
State Logic	→ contexts/, hooks/ (What data is stored and when it changes)
Data Access	→ services/

(How data is fetched from the backend)

Validation → `utils/authSchemas.js`
(What is valid input – separate from the form UI)

Configuration → `config/`
(Environment, HTTP client setup)

5. Reusable Components

What is it?

Build UI elements that can be used in multiple contexts without modification.

This project's reuse achievement

Component	Used In
Button	Every page (20+ uses)
Badge	SearchResultCard, MedicineCard, InventoryPage, AdminUsers, Dashboard
InfoCard	UserDashboard, PharmacyDashboard, AdminDashboard
Spinner	Route loading, button loading, page loading, auth checking
EmptyState	Search results, notifications, inventory
Modal	MedicineDetails, GenericRecommendation, AdminForms
Pagination	SearchResults, Inventory, AdminUsers, AdminMedicines

50+ components built. Zero duplication.

6. Component Composition

What is it?

Build complex UIs by combining simple components.

Example — SearchResultCard

Instead of one massive component with 300 lines:

```
// SearchResultCard is composed of smaller sub-components
function SearchResultCard({ medicine, ... }) {
  return (
    <article>
      <SmartBadges medicine={medicine} /> // ← sub-component
      <MedicineImagePlaceholder type={type} /> // ← sub-component
      <CardActions // ← sub-component
        onView={onView}
        onCompare={onCompare}
      />
    </article>
  )
}
```

Each sub-component has one job. They are easy to read, test, and modify.

7. Accessibility (A11y)

What is it?

Designing systems that can be used by people with disabilities.

Implemented in this project

Semantic HTML:

```
<nav aria-label="Breadcrumb">
  <main id="main-content">
    <article aria-label="Paracetamol 500mg – Best Value">
      <aside class="sidebar">
```

ARIA attributes:

```
<button aria-label="View details for Paracetamol" />
<button aria-pressed={isSaved} />
<input aria-label="Select medicine for comparison" />
<HiChevronRight aria-hidden="true" /> { /* Decorative icon */ }
```

Skip link (keyboard users):

```
<a href="#main-content" class="skip-link">Skip to main content</a>
```

Focus indicators:

```
className="focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-primary-500"
```

8. Responsive Design

What is it?

UI that works correctly on all screen sizes.

Strategy: Mobile-First

Tailwind classes are written for mobile first, then overridden for larger screens:

```
// 1 column on mobile → 2 on tablet → 3 on desktop → 4 on large desktop
className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4"

// Sidebar hidden on mobile, shown on large screens
className="hidden lg:flex"
```

All pages, cards, tables, forms, and navigation are tested at:

- 375px (mobile)
- 768px (tablet)
- 1024px (laptop)
- 1280px (desktop)

- 1536px (large desktop)

9. Error Handling

Multiple Layers

Layer	What It Handles
AppErrorBoundary	Catches any uncaught React crash
AuthContext	Catches auth errors, stores in authError state
axiosClient interceptors	Catches 401 globally
ErrorState component	Shows friendly error with retry button
OfflinePage	Shows when browser is offline
SessionExpiredPage	Shows when JWT expires
NotFoundPage	Shows for unknown URLs
UnauthorizedPage	Shows for wrong role access

10. Performance Engineering

Techniques Used

Lazy Loading (Code Splitting)

```
const AdminDashboard = lazy(() => import('../pages/admin/AdminDashboard'))
// AdminDashboard code is NOT loaded until user visits /admin/dashboard
```

Debouncing

```
const debouncedQuery = useDebounce(searchQuery, 400)
// API call only fires 400ms after the user stops typing
// Without this: API called on every single keystroke
```

Memoisation

```
const login = useCallback(async (credentials) => { ... }, [])
// login function is not recreated on every render
```

React Query Caching

```
staleTime: 60000 // Data cached for 1 minute
gcTime: 300000 // Cache kept for 5 minutes
```

Visiting the same medicine page twice: second visit is instant (cached).

11. Scalability

Horizontal Scalability (adding features)

Adding a new user role (e.g., "nurse"):

1. Add `NURSE: 'nurse'` to `USER_ROLES` in `constants/app.js`
2. Create `NurseLayout.jsx` copying pattern from `UserLayout.jsx`
3. Create `NURSE_NAV` in `navConfig.js`
4. Add routes in `AppRouter.jsx`
5. Existing code untouched

Adding a new admin page:

1. Create `pages/admin/AdminNewPage.jsx`
2. Add route in `AppRouter.jsx`
3. Add nav item in `navConfig.js`
4. Add route constant in `routes.js`

12. Maintainability

What makes this project easy to maintain

Feature	Benefit
Consistent naming conventions	New developers can find files intuitively
Barrel exports	Change internal file structure without breaking imports
TODO comments	Backend integration points are explicit and easy to find
Design tokens	Visual changes propagate from one source
Route constants	Refactoring URLs is safe (catch errors at import time)
JSDoc comments	Every major component and function is documented

TODO Pattern

Every backend integration point has a clear comment:

```
// TODO: replace mock with → const { data } = await authService.login(credentials)
// TODO: price from GET /api/v1/medicines/:id
// TODO: call userService.updateMe(data) then setUserProfile(response.data)
```

When backend is ready: search for `TODO`: → implement line by line.